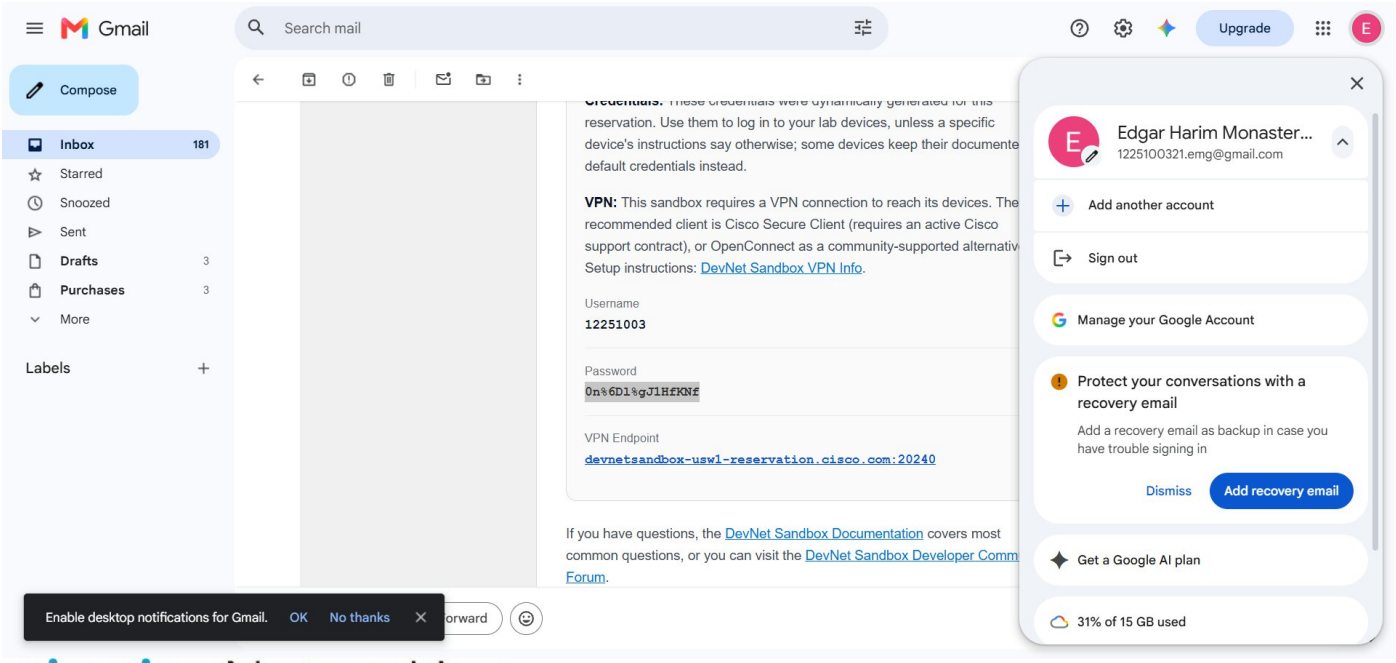


Networking Academy

Lab – RESTCONF with Python

Objectives

Part 1: RESTCONF basics in Python

Part 2: Modify interface configuration with RESTCONF in Python

Background / Scenario

Following up the previous lab activity, in this lab you will learn how to execute the RESTCONF API calls using Python scripts.

Required Resources

- Python 3.x environment
- Access to a router with the IOS XE operating system version 16.6 or higher.

Instructions

Part 1: RESTCONF in Python

In this part, you will use Python to request a RESTCONF API.

Step 1: Import modules and disable SSL warnings.

a. In IDLE, click **File > New File** to open IDLE Editor.

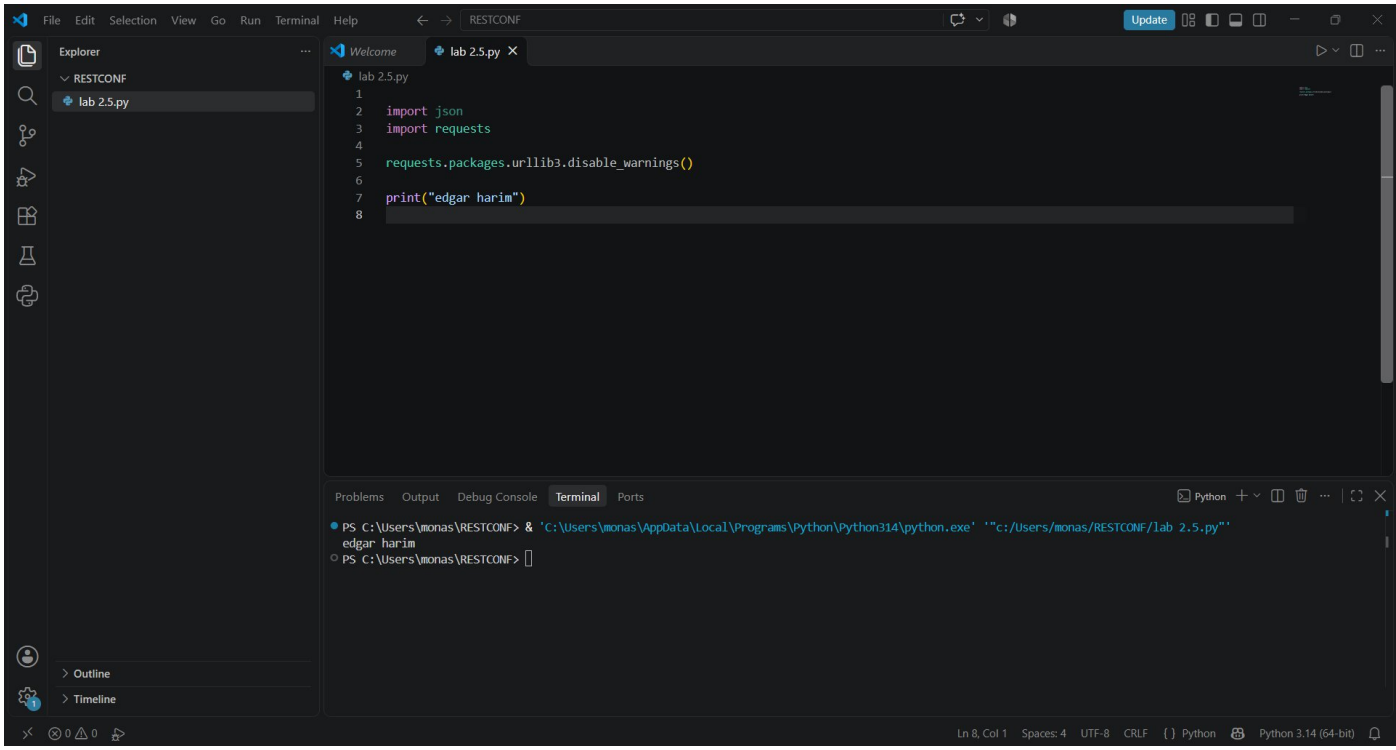
b. Save the file as **lab 2.5.py**.

c. Enter the following commands to import the modules and disable SSL certificate warnings:

```
import json
import requests
requests.packages.urllib3.disable_warnings()
```

©2017-2026Ciscoand/oritsaffiliates.Allrightsreserved.CiscoPublic

The **json** module includes methods convert JSON data to Python objects and vice versa. The **requests** module has methods that will let us send REST requests to a URI.



Step 2: Build the request components.

Create a string variable to hold the API endpoint URI and two dictionaries, one for the request header and one for the body JSON. These are the same tasks you completed in the Postman application.

d. Create a variable named **api_url** and assign the URL (adjust the IP address to match the router's current address).

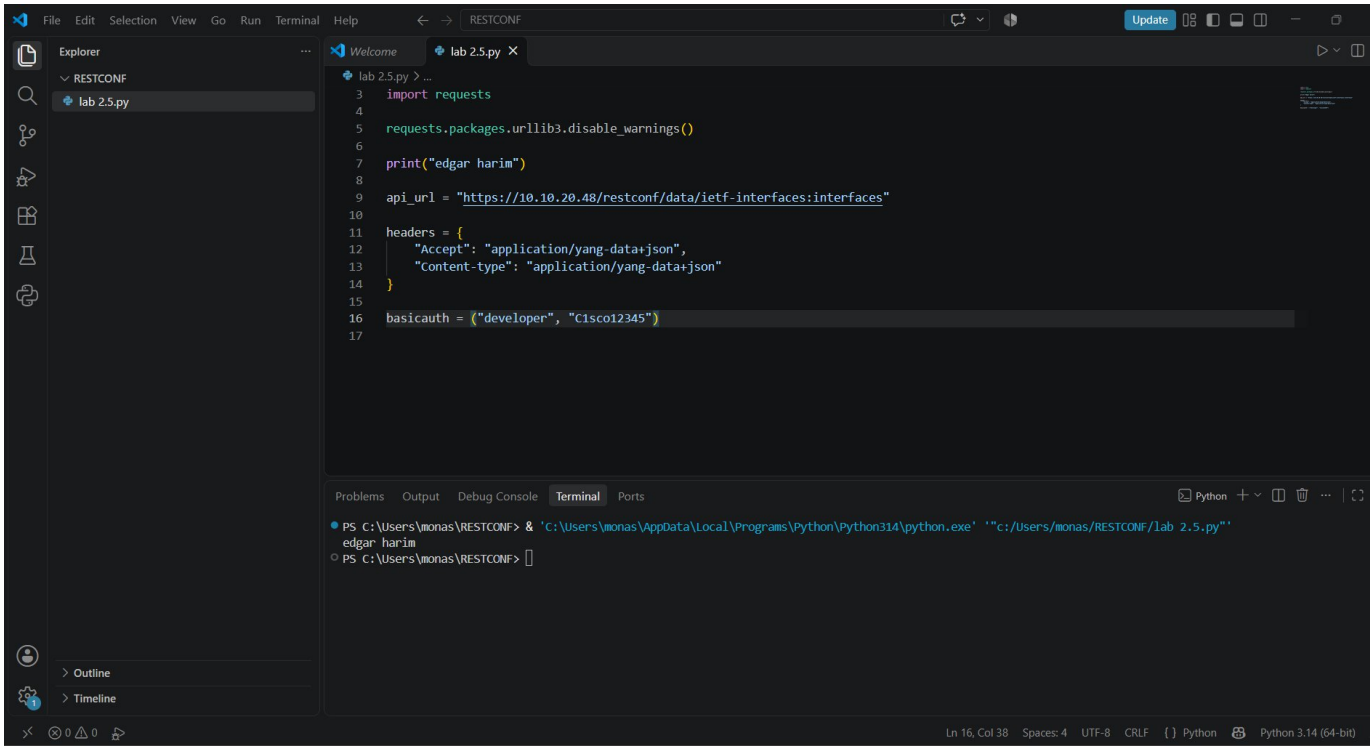
```
api_url = "https://192.168.56.101/restconf/data/ietf-interfaces:interfaces"
```

e. Create a dictionary variable named **headers** that has keys for **Accept** and **Content-type** and assign the keys the value **application/yang-data+json**.

```
headers = { "Accept": "application/yang-data+json",
            "Content-type":"application/yang-data+json"
          }
```

f. Create a Python tuple variable named **basicauth** that has two keys needed for authentication, **username** and **password**.

```
basicauth = ("cisco", "cisco123!")
```



Step 3: Send the request.

You will now use the variables created in the previous step as parameters for the **requests.get()** method. This method sends an HTTP GET request to the RESTCONF API. You will assign the result of the request to a variable name **resp**. That variable will hold the JSON response from the API. If the request is successful, the JSON will contain the returned YANG data model.

g. Enter the following statement:

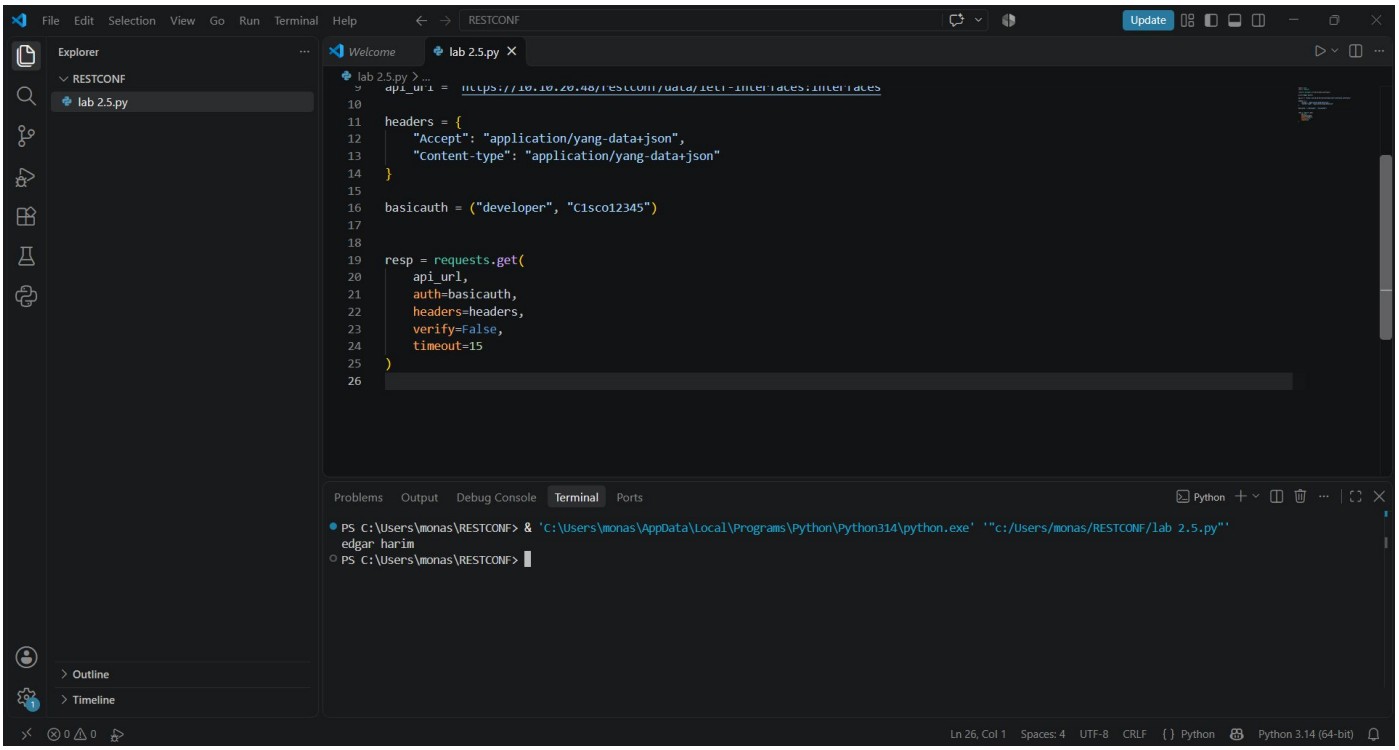
```
resp = requests.get(api_url, auth=basicauth, headers=headers, verify=False)
```

The various elements of this statement are:

Element	Explanation
`resp`	the variable to hold the response from the API.
`requests.get()`	the method that actually makes the GET request.
`api_url`	the variable that holds the URL address string
`auth`	the tuple variable created to hold the authentication information
`headers=headers`	a parameter that is assigned to the headers variable
`verify=False`	disables verification of the SSL certificate when the request is made

h. Save your script and run it. There will not be any output yet but the script should run without errors. If not, review the steps and make sure your code does not contain any errors.

©2017-2026Ciscoand/oritsaffiliates.Allrightsreserved.CiscoPublic



Step 4: Evaluate the response.

Now the YANG model response values can be extracted from the response JSON.

i. The response JSON is not compatible with Python dictionary and list objects so it is converted to Python format. Create a new variable called **response_json** and assign the variable **resp** to it adding the **json()** method to convert the JSON. The statement is as follows:

```
response_json = resp.json()
```

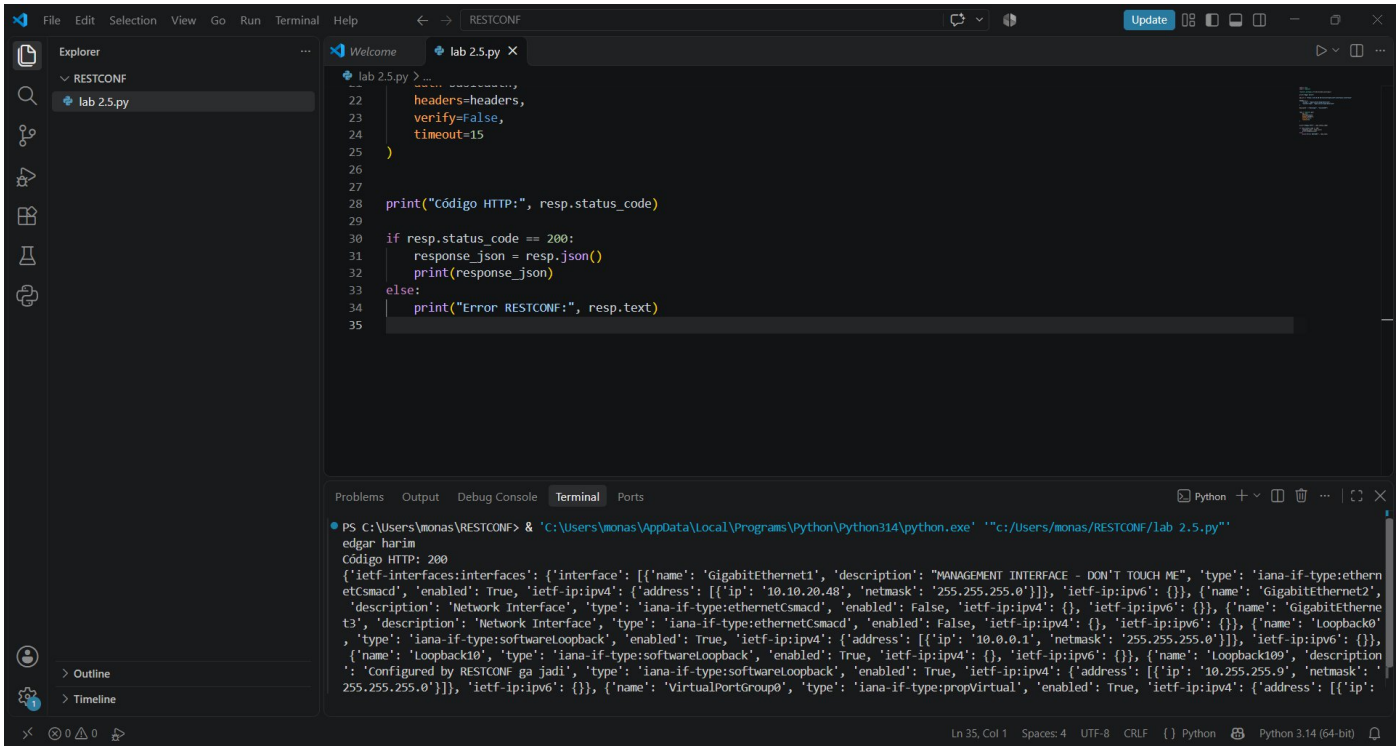
j. You can verify that your code returns the JSON in the IDLE Shell by temporarily adding a print statement to your script, as follows:

```
print(response_json)
```

k. Save and run your script. You should get output similar to the following although your service ticket number will be different:

```
>>>
 RESTART: B:/DevNetAcad PS-E/Workshops/ETW3 Model Driven Programmability/Python Files with Solutions/idle/lab 2.9.py
{'ietf-interfaces:interfaces': {'interface': [{'name': 'GigabitEthernet1', 'type': 'iana-if-type:ethernetCsmacd', 'enabled':
True, 'ietf-ip:ipv4': {}, 'ietf-ip:ipv6': {}}, {'name': 'Loopback1', 'description': 'WHATEVER', 'type': 'iana-if-type:softwar
eLoopback', 'enabled': True, 'ietf-ip:ipv4': {'address': [{'ip': '2.2.2.2', 'netmask': '255.255.255.0'}]}, 'ietf-ip:ipv6': {}
}, {'name': 'Loopback2', 'description': 'NEWBUTSAME', 'type': 'iana-if-type:softwareLoopback', 'enabled': True, 'ietf-ip:ipv4
': {}, 'ietf-ip:ipv6': {}}, {'name': 'Loopback99', 'description': 'WHATEVER99', 'type': 'iana-if-type:softwareLoopback', 'ena
bled': True, 'ietf-ip:ipv4': {'address': [{'ip': '99.99.99.99', 'netmask': '255.255.255.0'}]}, 'ietf-ip:ipv6': {}}, {'name':
'Loopback100', 'description': 'TEST1', 'type': 'iana-if-type:softwareLoopback', 'enabled': True, 'ietf-ip:ipv4': {'address':
[{'ip': '100.100.100.100', 'netmask': '255.255.255.0'}]}, 'ietf-ip:ipv6': {}}]}
>>>
```

©2017-2026Ciscoand/oritsaffiliates.Allrightsreserved.CiscoPublic

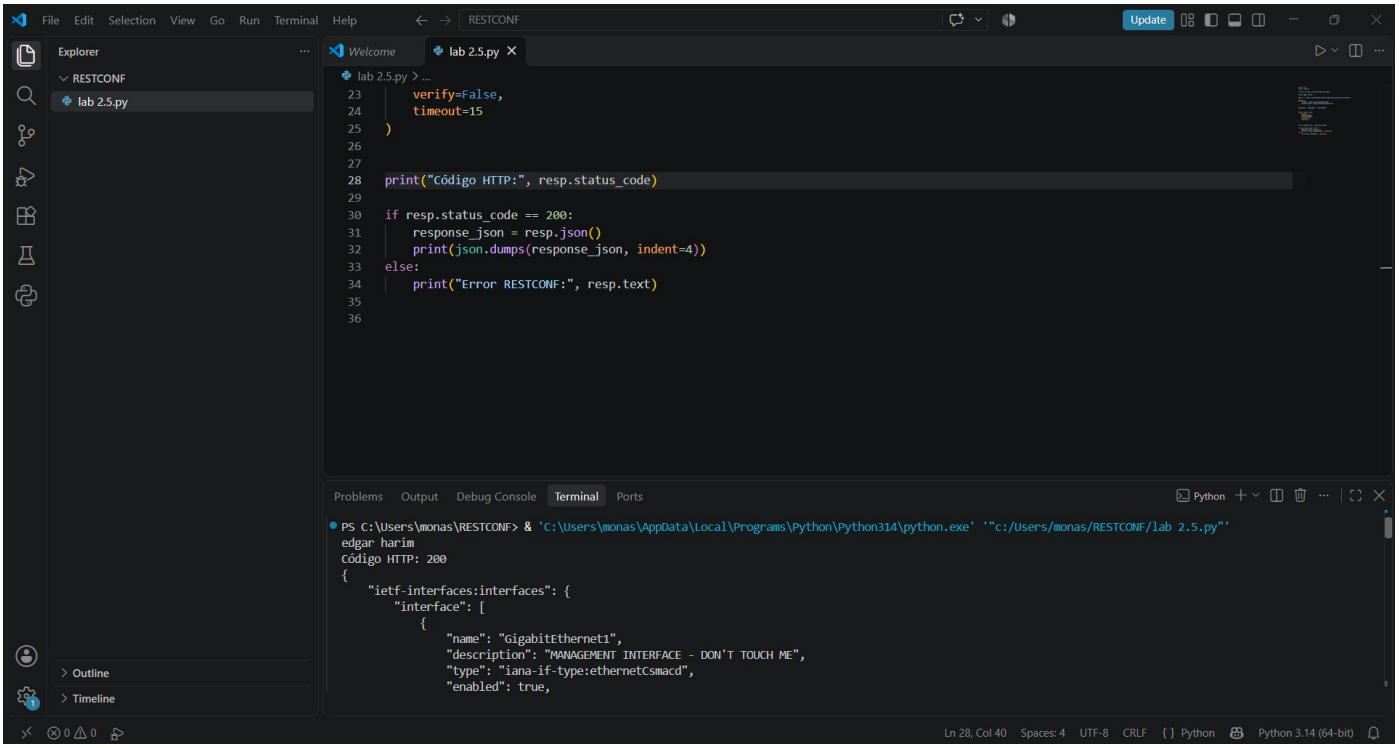


I. To prettify the output, use the json.dumps() function with the “indent” parameter:

```
print(json.dumps(response_json, indent=4))
```

©2017-2026Ciscoand/oritsaffiliates.Allrightsreserved.CiscoPublic

m. Save and run your script. If you experience errors, check the code again.



Part 2: Modify interface configuration with RESTCONF in Python

Step 5: Create the Python HTTP PUT request

In this part, you will use Python to request a RESTCONF API with a PUT method to create or modify existing configuration.

Step 6: Import modules and disable SSL warnings.

n. In IDLE, click **File > New File** to open IDLE Editor.

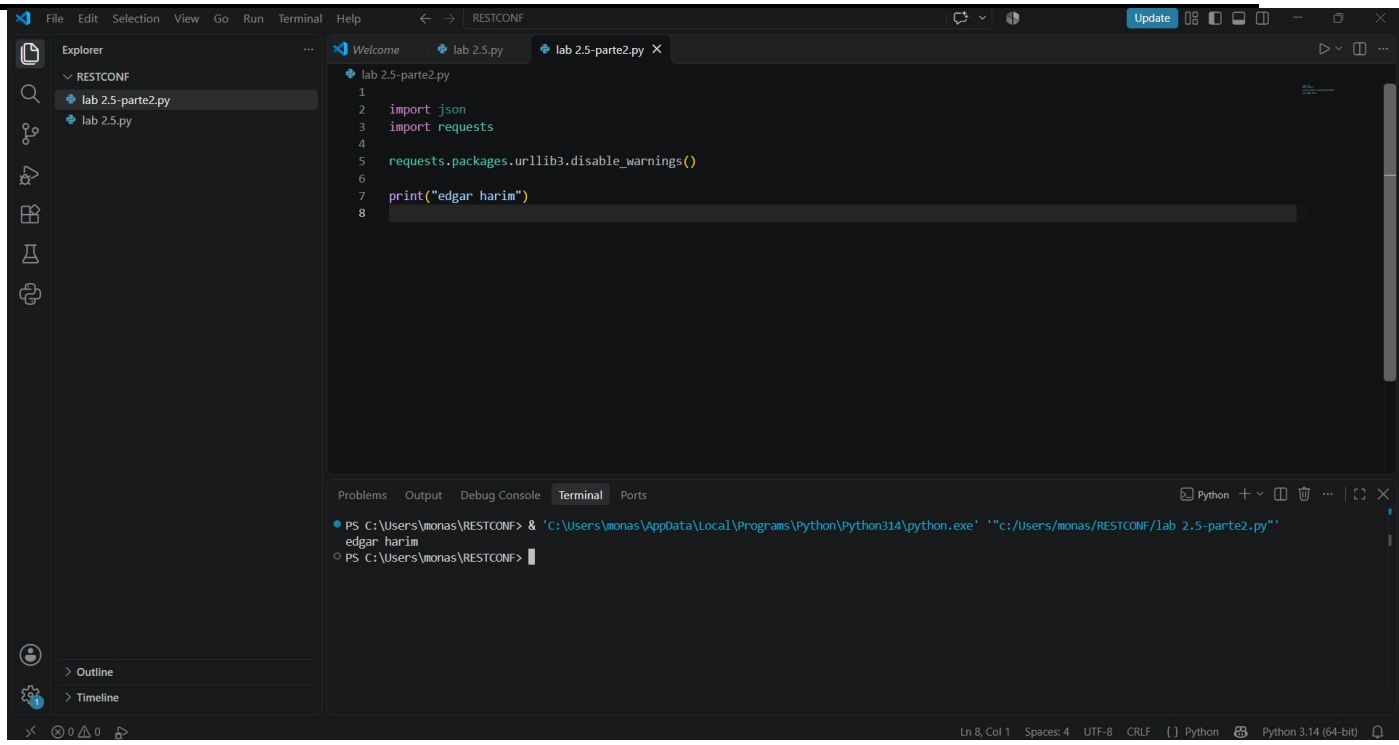
o. Save the file as **lab 2.5 part2.py**.

p. Enter the following commands to import the modules and disable SSL certificate warnings:

```
import json
import requests
requests.packages.urllib3.disable_warnings()
```

The **json** module includes methods convert JSON data to Python objects and vice versa. The **requests** module has methods that will let us send REST requests to a URI.

©2017-2026Ciscoand/oritsaffiliates.Allrightsreserved.CiscoPublic



Step 7: Build the request components.

Create a string variable to hold the API endpoint URI and two dictionaries, one for the request header and one for the body JSON. These are the same tasks you completed in the Postman application.

q. Create a variable named **api_url** and assign the URL that targets the **Loopback99** interface.

```
api_url = "https://192.168.56.101/restconf/data/ietf-
interfaces:interfaces/interface=Loopback99"
```

r. Create a dictionary variable named **headers** that has keys for **Accept** and **Content-type** and assign the keys the value **application/yang-data+json**.

```
headers = {
            "Accept": "application/yang-data+json",
            "Content-type":"application/yang-data+json"
          }
```

s. Create a Python tuple variable named **basicauth** that has two keys needed for authentication, **username** and **password**.

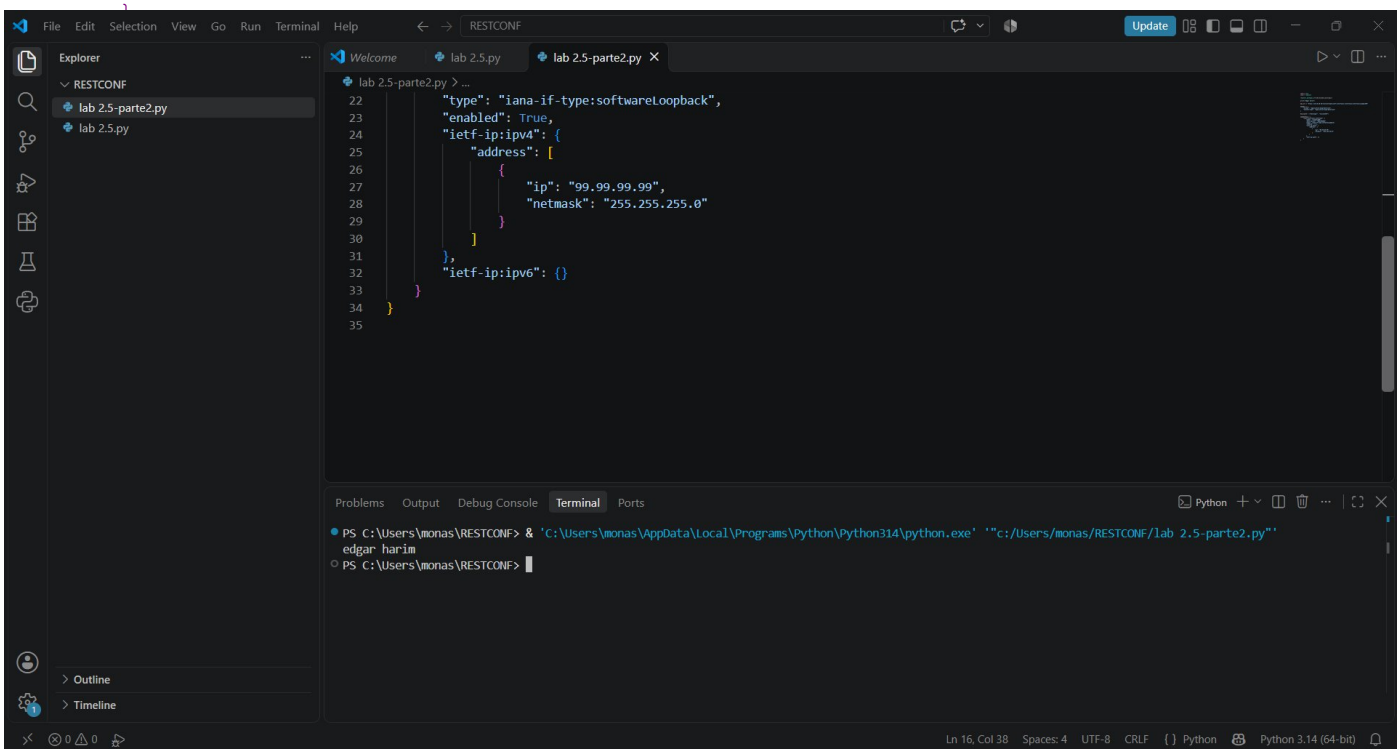
```
basicauth = ("cisco", "cisco123!")
```

t. Create a Python dictionary variable yangConfig holding the YANG data to create new interface Loopback99 (you use here the dictionary data from the Postman lab before, be aware that the JSON's boolean **true** is in Python **True** with capital "T"):

```
yangConfig = {
    "ietf-interfaces:interface":
        { "name": "Loopback99",
        "description": "WHATEVER99",
        "type": "iana-if-type:softwareLoopback",
        "enabled": True,
        "ietf-ip:ipv4":
            { "address": [
```

©2017-2026Ciscoand/oritsaffiliates.Allrightsreserved.CiscoPublic

```
                {
                    "ip": "99.99.99.99",
                    "netmask": "255.255.255.0"
                }
            ]
        },
        "ietf-ip:ipv6": {}
    }
```



Step 8: Send the PUT request.

You will now use the variables created in the previous step as parameters for the **requests.put()** method. This method sends an HTTP PUT request to the RESTCONF API. You will assign the result of the request to a variable name **resp**. That variable will hold the JSON response from the API. If the request is successful, the JSON will contain the returned YANG data model.

u. Enter the following statement:

```
resp = requests.put(api_url, data=json.dumps(yangConfig), auth=basicauth,
headers=headers, verify=False)
if(resp.status_code >= 200 and resp.status_code <= 299):
    print("STATUS OK: {}".format(resp.status_code))
else:
    print("Error code {}, reply: {}".format(resp.status_code, resp.json()))
```

The various elements of this statement are:

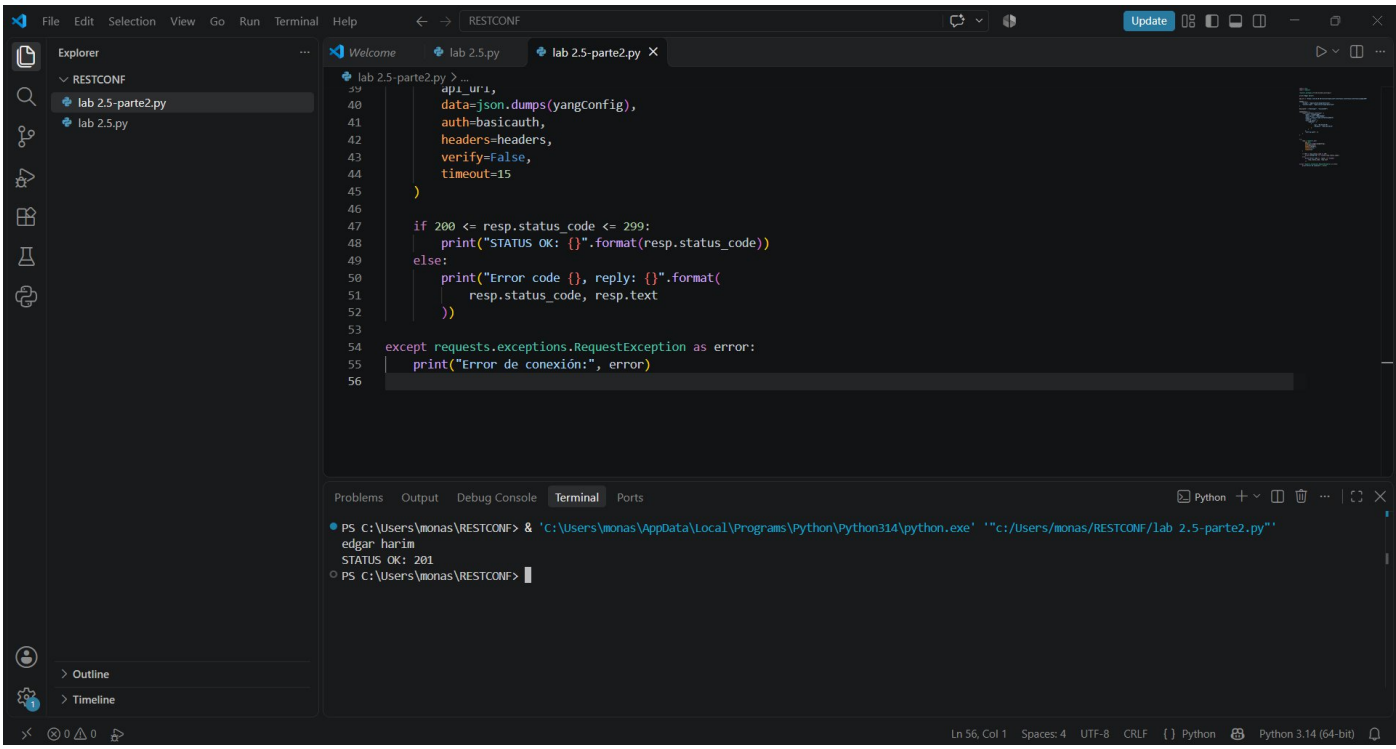
Element	Explanation
`resp`	the variable to hold the response from the API.
`requests.get()`	the method that actually makes the GET request.

©2017-2026Ciscoand/oritsaffiliates.Allrightsreserved.CiscoPublic

`api_url`	the variable that holds the URL address string
`data`	the data to the sent to the API endpoint
`auth`	the tuple variable created to hold the authentication information
`headers=headers`	a parameter that is assigned to the headers variable
`verify=False`	disables verification of the SSL certificate when the request is made
`resp.status_code`	The HTTP status code in the API Request reply

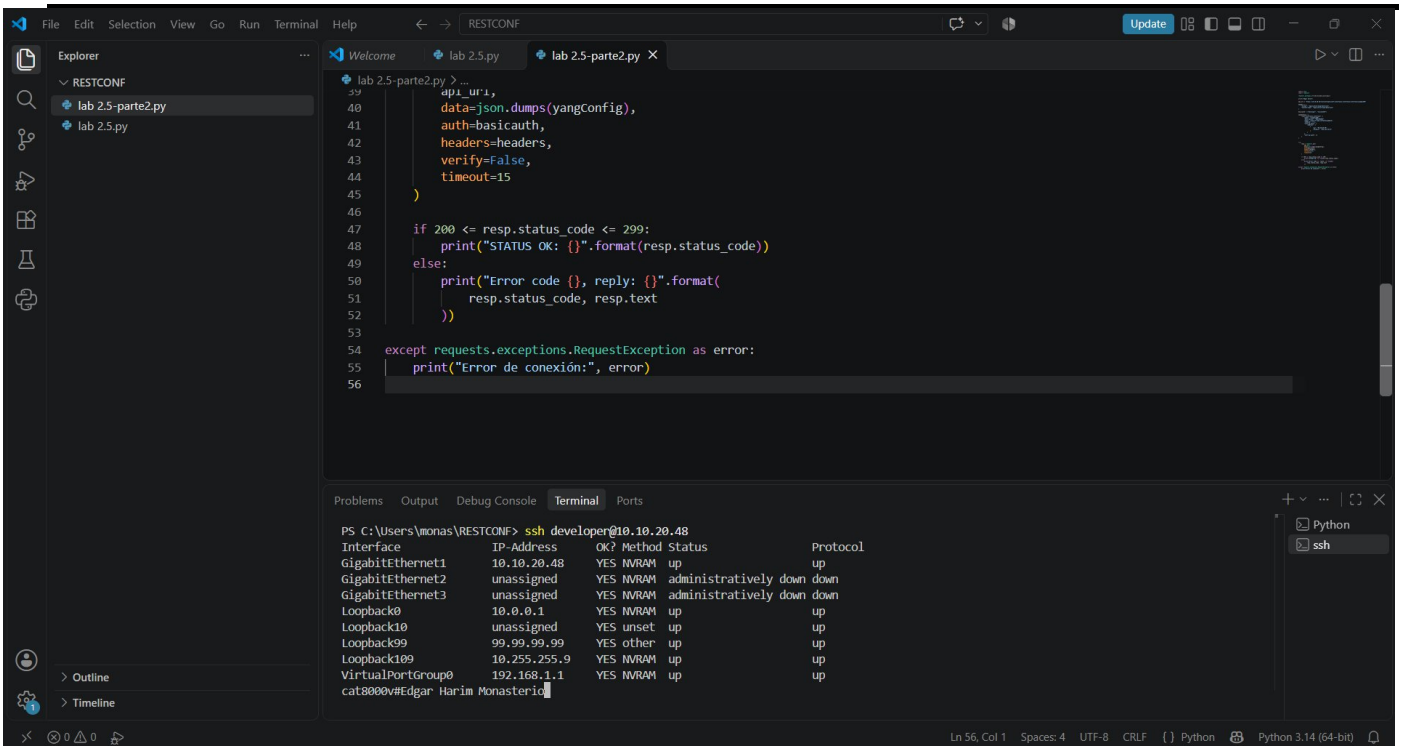
v. Save your script and run it.

```
>>>
 RESTART: B:/DevNetAcad PS-E/Workshops/ETW3 Model Driven Programmability/Python Files with Solutions/idle/lab 2.9 part2.py
STATUS OK: 201
>>>
```

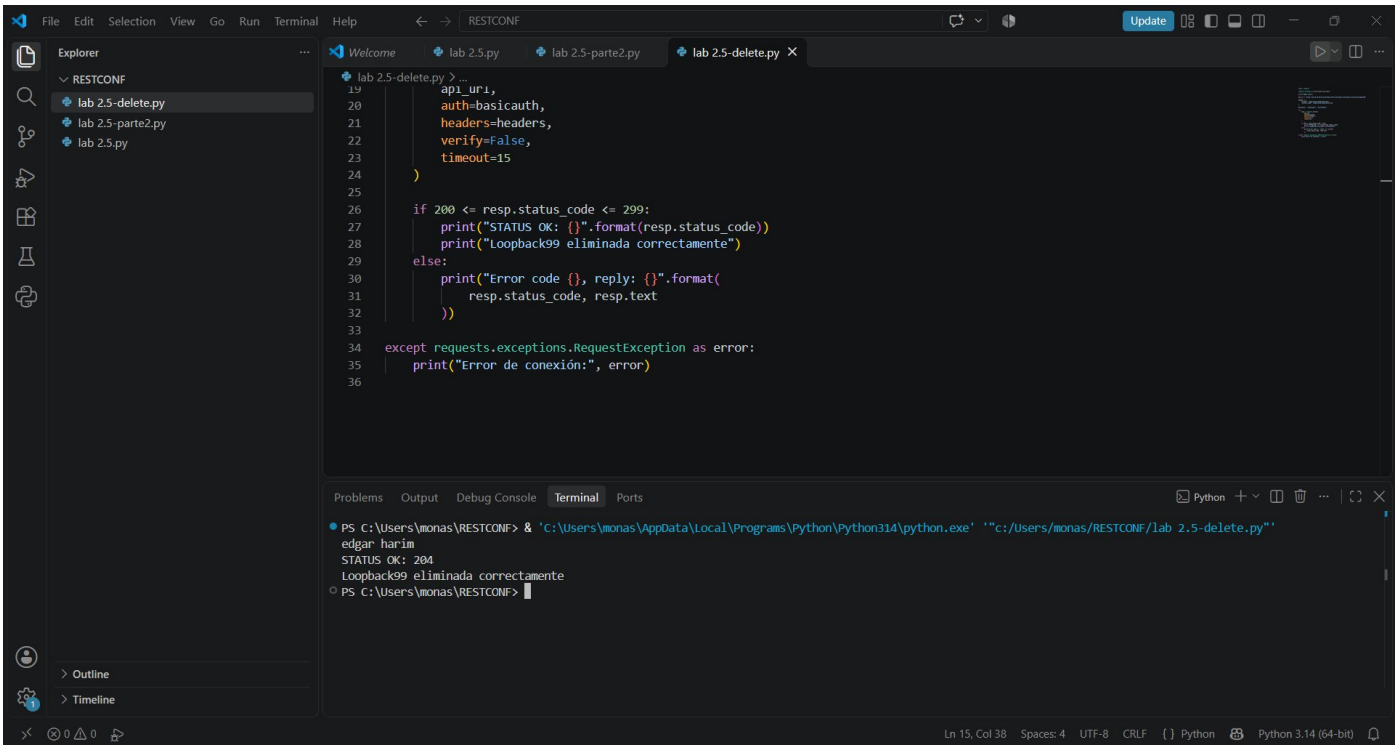


w. erify using the IOS CLI that the new Loopback99 interface has been created (sh ip int brief).

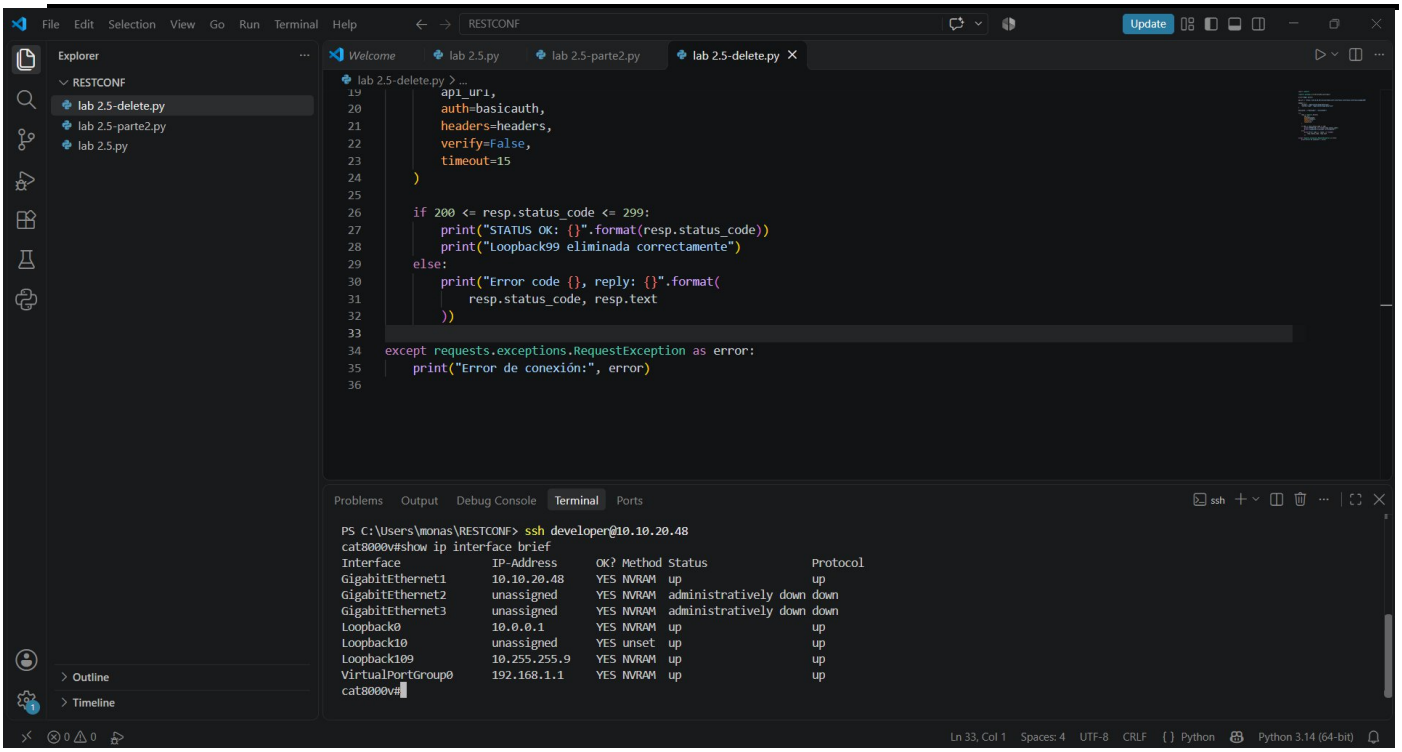
©2017-2026Ciscoand/oritsaffiliates.Allrightsreserved.CiscoPublic



x. Modify the code to delete the interface Loopback99.



©2017-2026Ciscoand/oritsaffiliates.Allrightsreserved.CiscoPublic



y. What changes were applied to the code to delete the interface Loopback99?

cambie requests.put() por requests.delete() manteniendo la dirección restconf, las credenciales y los encabezados, al ejecutar el router respondio y comprobe que la interfaz ya no aparecia

©2017-2026Ciscoand/oritsaffiliates.Allrightsreserved.CiscoPublic

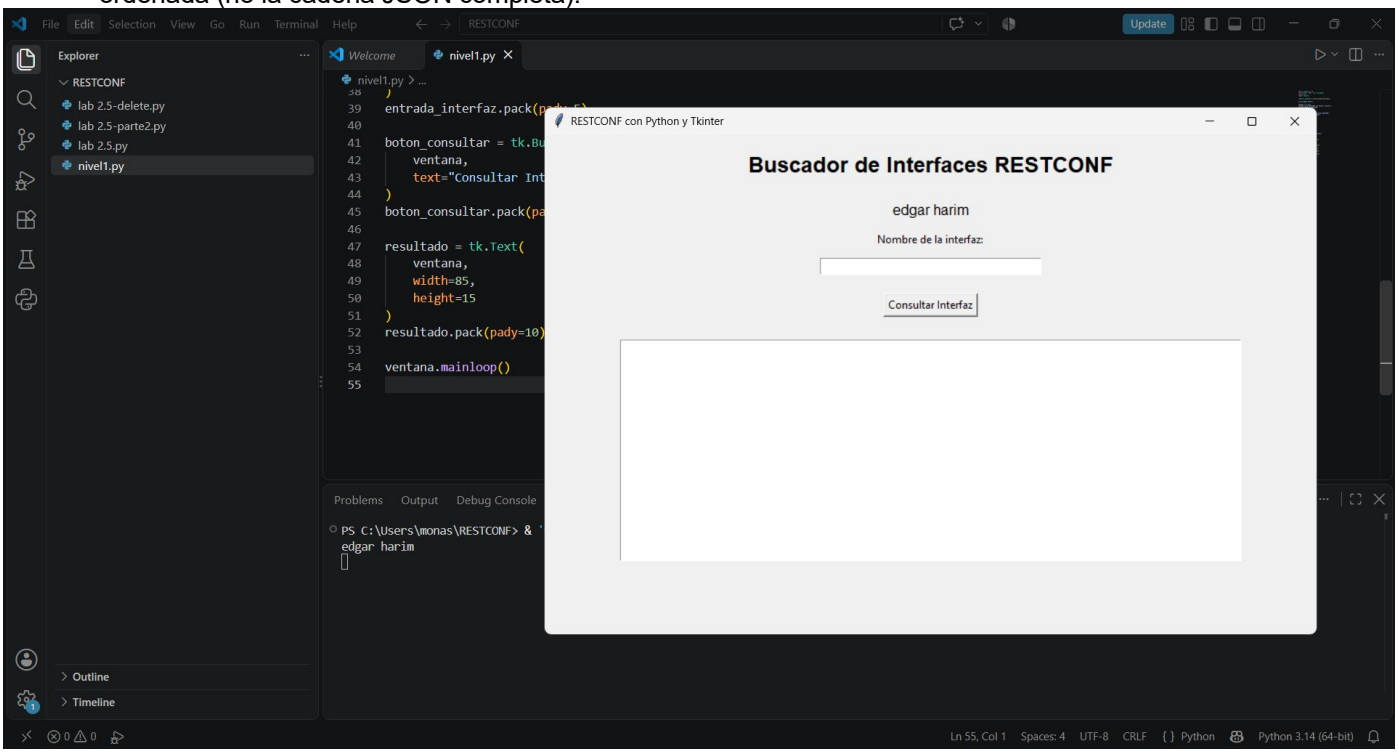
Nivel 1: Construcción de la Interfaz y Consulta (GET)

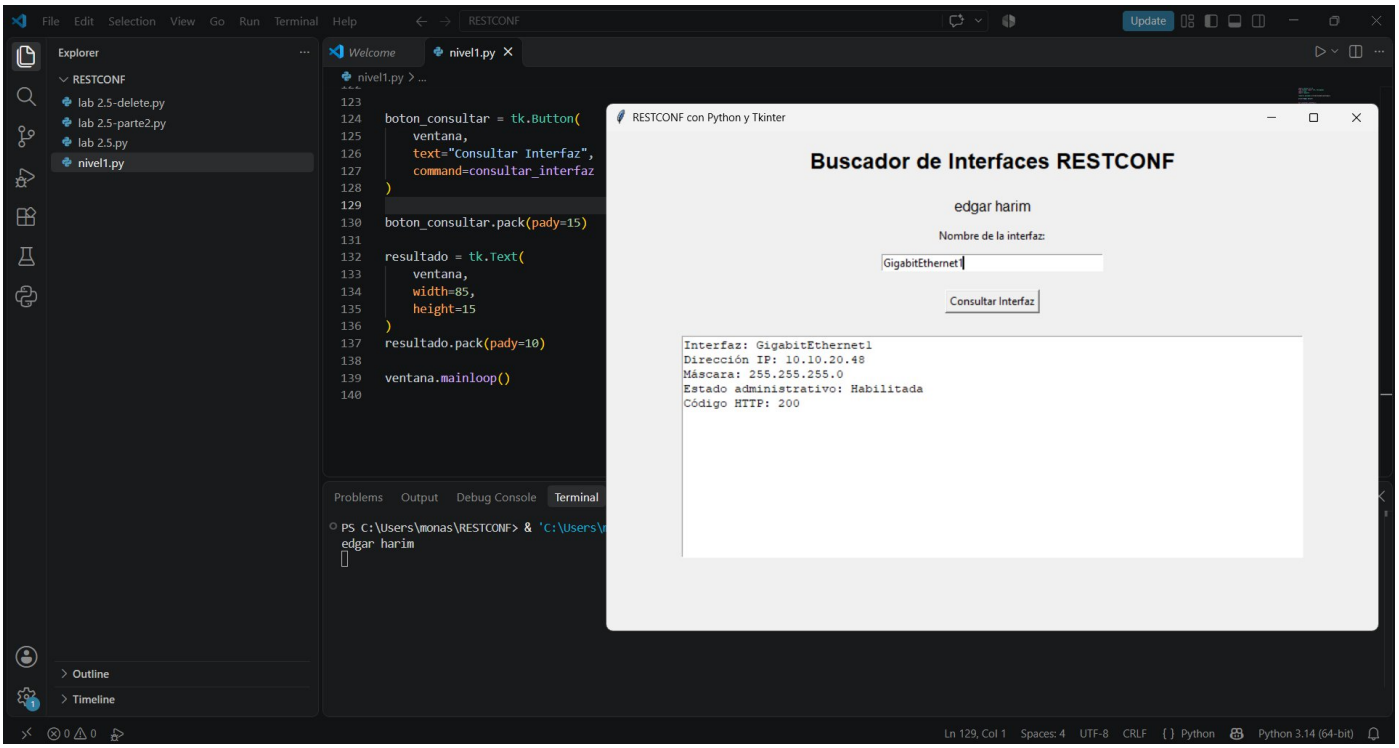
Desafio 1: Buscador de Interfaces con Formato Visual

Objetivo: Diseñar la estructura básica del cliente RESTCONF con Tkinter.

Tarea: Crear una ventana que incluya:

1. Un campo de entrada (tk.Entry) para escribir el nombre de la interfaz (ej. Loopback99, GigabitEthernet1).
2. Un botón (tk.Button) para "Consultar Interfaz" que ejecute requests.get().
3. Un area de texto (tk. Text o ttk.Treeview) para mostrar la IP, la mascara y el estado de la interfaz de forma ordenada (no la cadena JSON completa).





©2017-2026Ciscoand/oritsaffiliates.Allrightsreserved.CiscoPublic

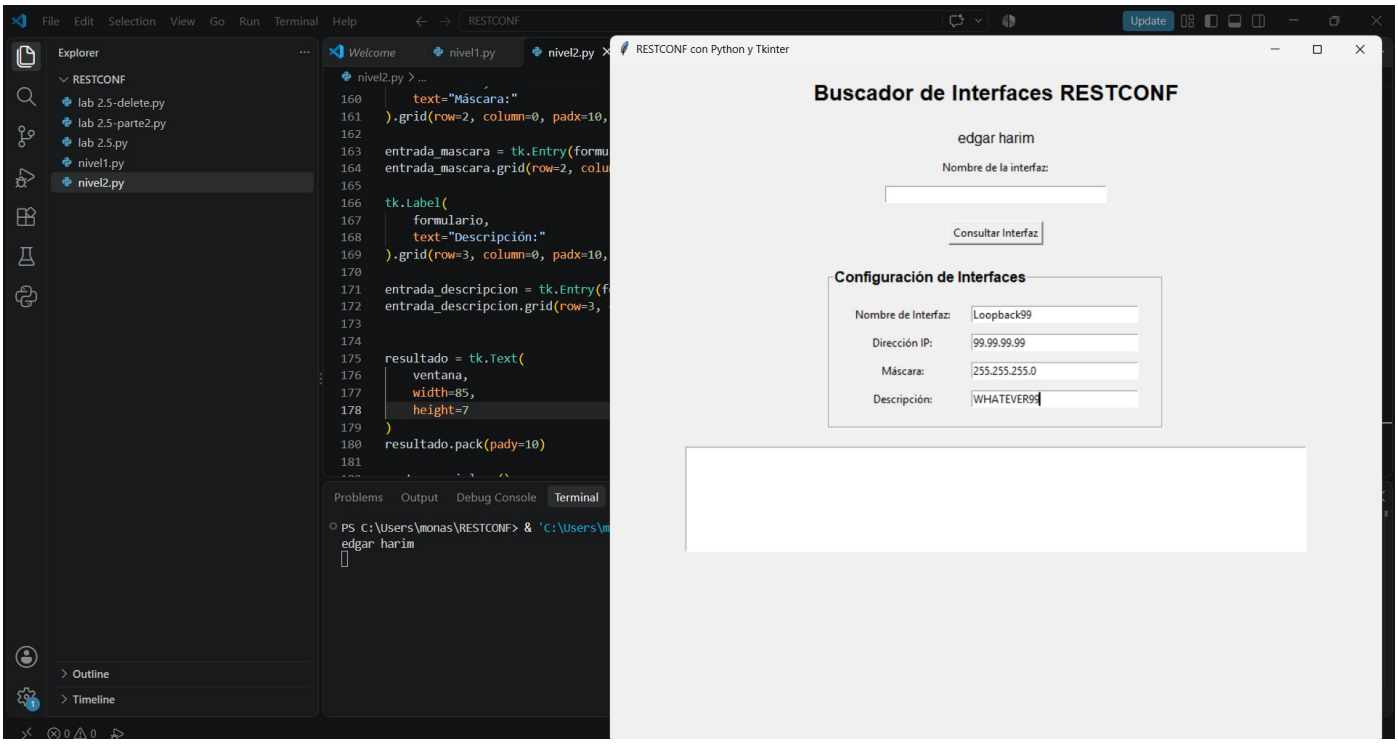
Nivel 2: Formularios de Configuración (PUT/POST/ DELETE)

Desafio 2: Formulario de Creacion de Interfaces

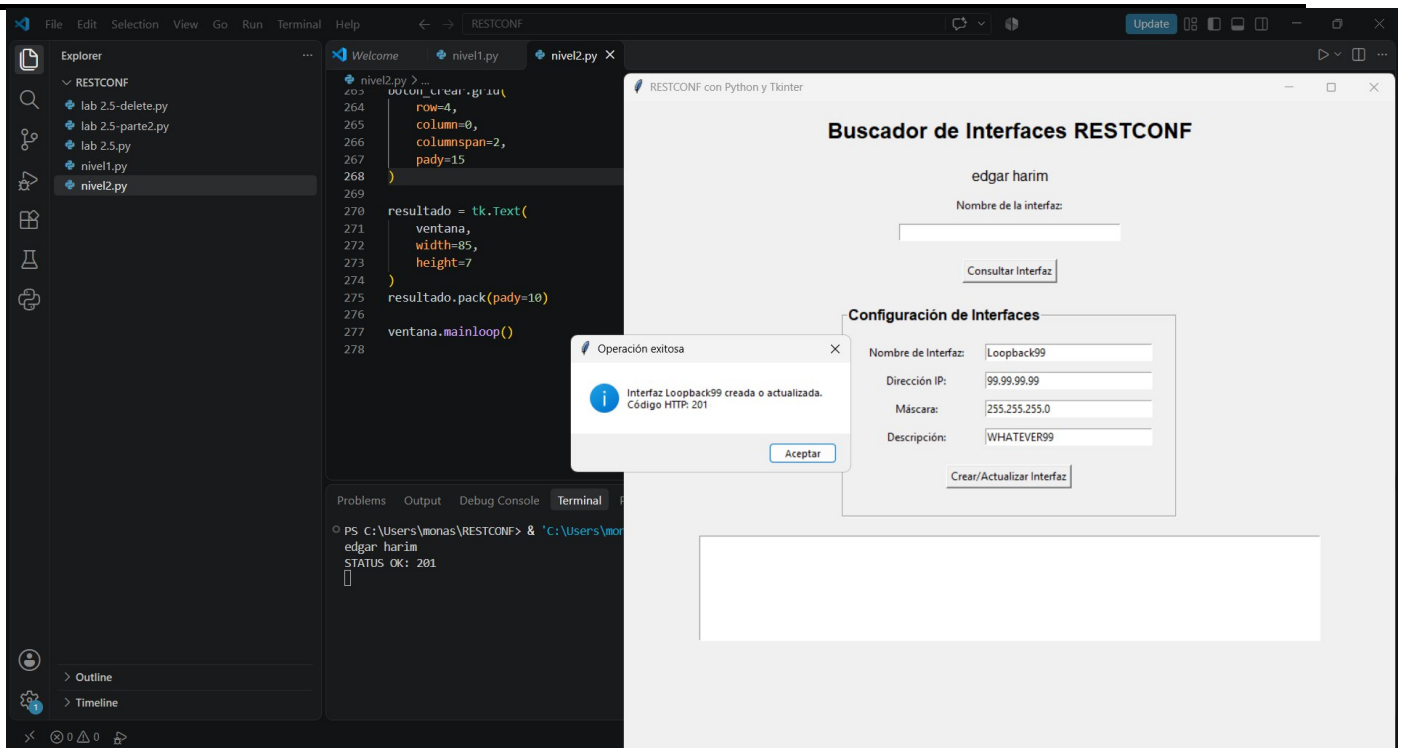
Objetivo: Recopilar datos desde un formulario de Tkinter y empaquetarlos en un diccionario yangConfig para enviar mediante requests.put().

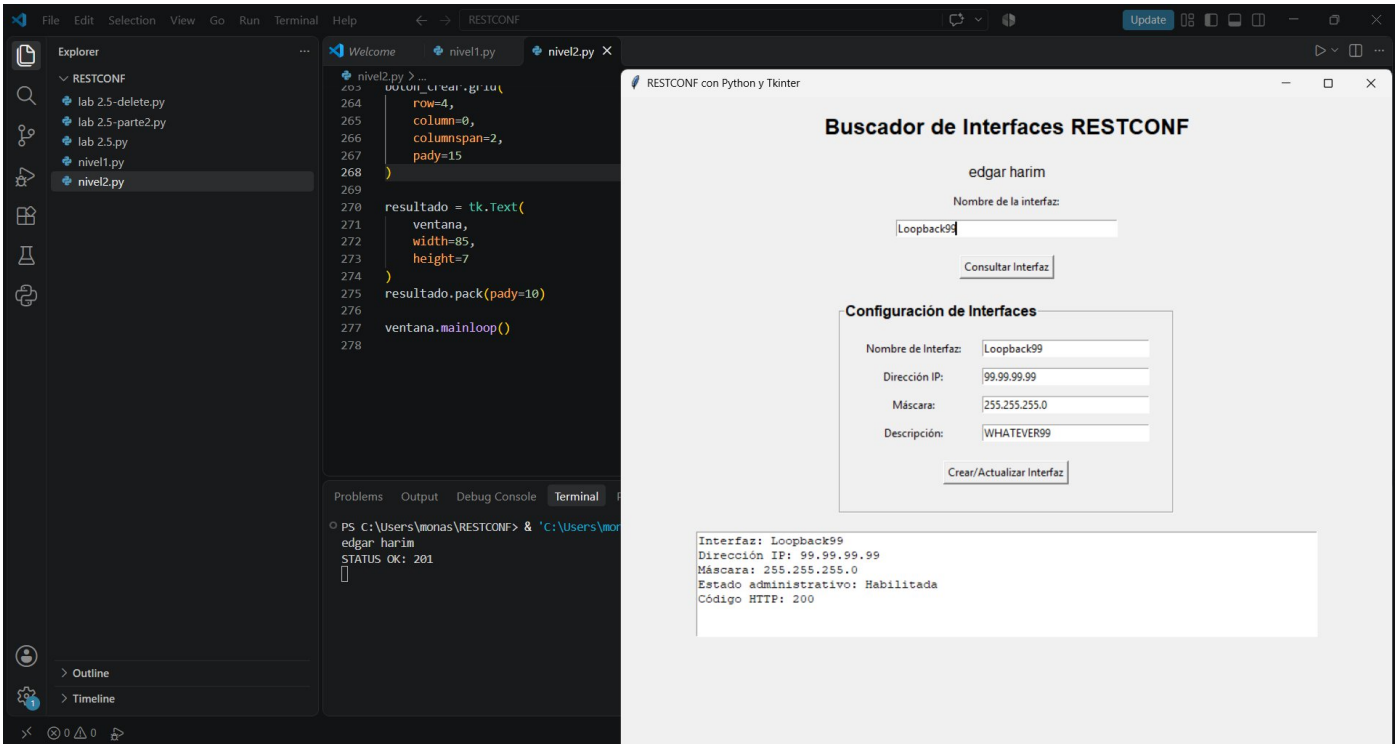
1. Tarea: Agregar campos para Nombre de Interfaz, Dirección IP, Máscara y Descripción. Al presionar un botón "Crear/Actualizar Interfaz":

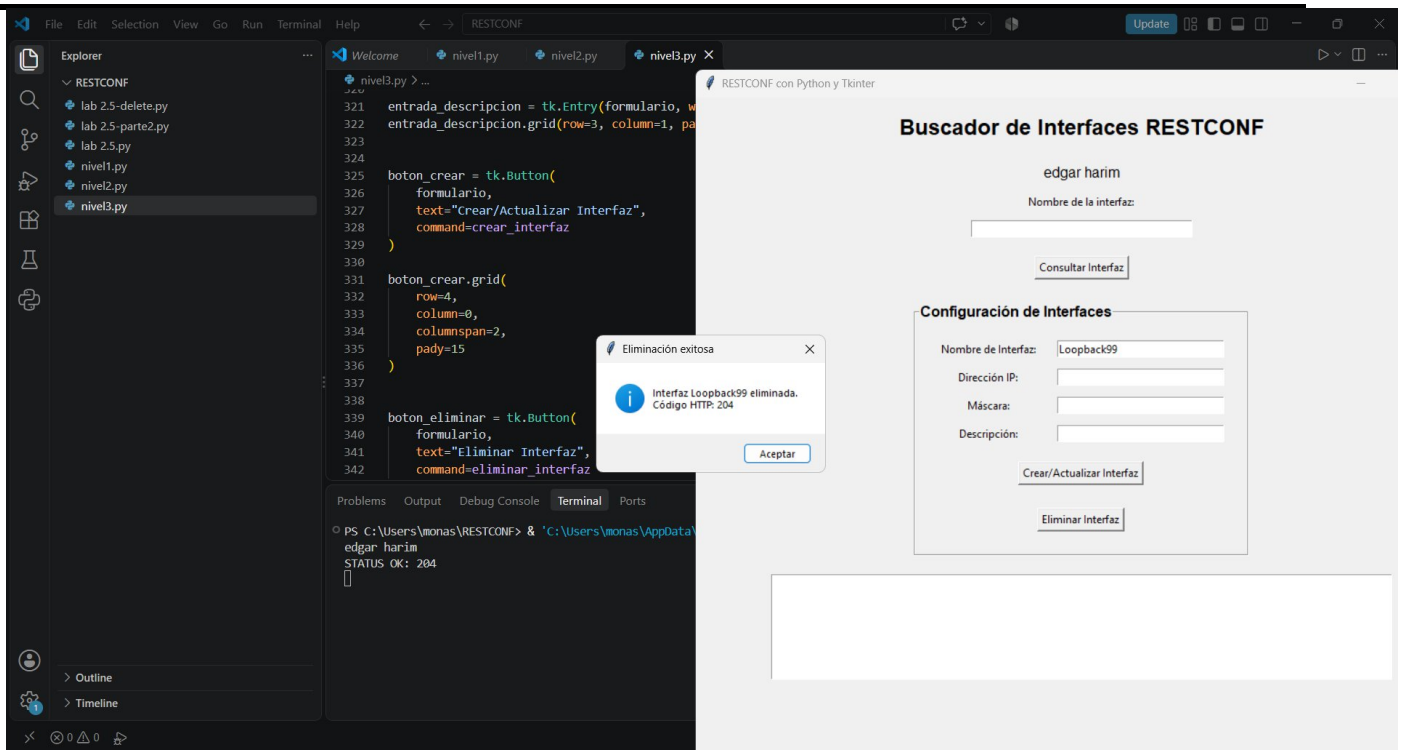
- Se arma dinamicamente el JSON.
- Si la API responde 201 Created o 200 OK, se muestra una ventana emergente de exito (messagebox.showinfo).



©2017-2026Ciscoand/oritsaffiliates.Allrightsreserved.CiscoPublic


RESTCONF con Python y Tkinter
Buscador de Interfaces RESTCONF
edgar harim
Nombre de la interfaz:
Consultar Interfaz
Configuración de Interfaces
Nombre de Interfaz: Loopback99
Dirección IP: 99.99.99.99
Máscara: 255.255.255.0
Descripción: WHATEVER99
Crear/Actualizar Interfaz
Operación exitosa
Interfaz Loopback99 creada o actualizada.
Código HTTP: 201
Aceptar
PS C:\Users\monas\RESTCONF> & 'C:\Users\mon
edgar harim
STATUS OK: 201


RESTCONF con Python y Tkinter
Buscador de Interfaces RESTCONF
edgar harim
Nombre de la interfaz:
Loopback99
Consultar Interfaz
Configuración de Interfaces
Nombre de Interfaz: Loopback99
Dirección IP: 99.99.99.99
Máscara: 255.255.255.0
Descripción: WHATEVER99
Crear/Actualizar Interfaz
Interfaz: Loopback99
Dirección IP: 99.99.99.99
Máscara: 255.255.255.0
Estado administrativo: Habilitada
Código HTTP: 200
PS C:\Users\monas\RESTCONF> & 'C:\Users\mon
edgar harim
STATUS OK: 201



Nivel 3: Manejo de Excepciones y Experiencia de Usuario (UX)

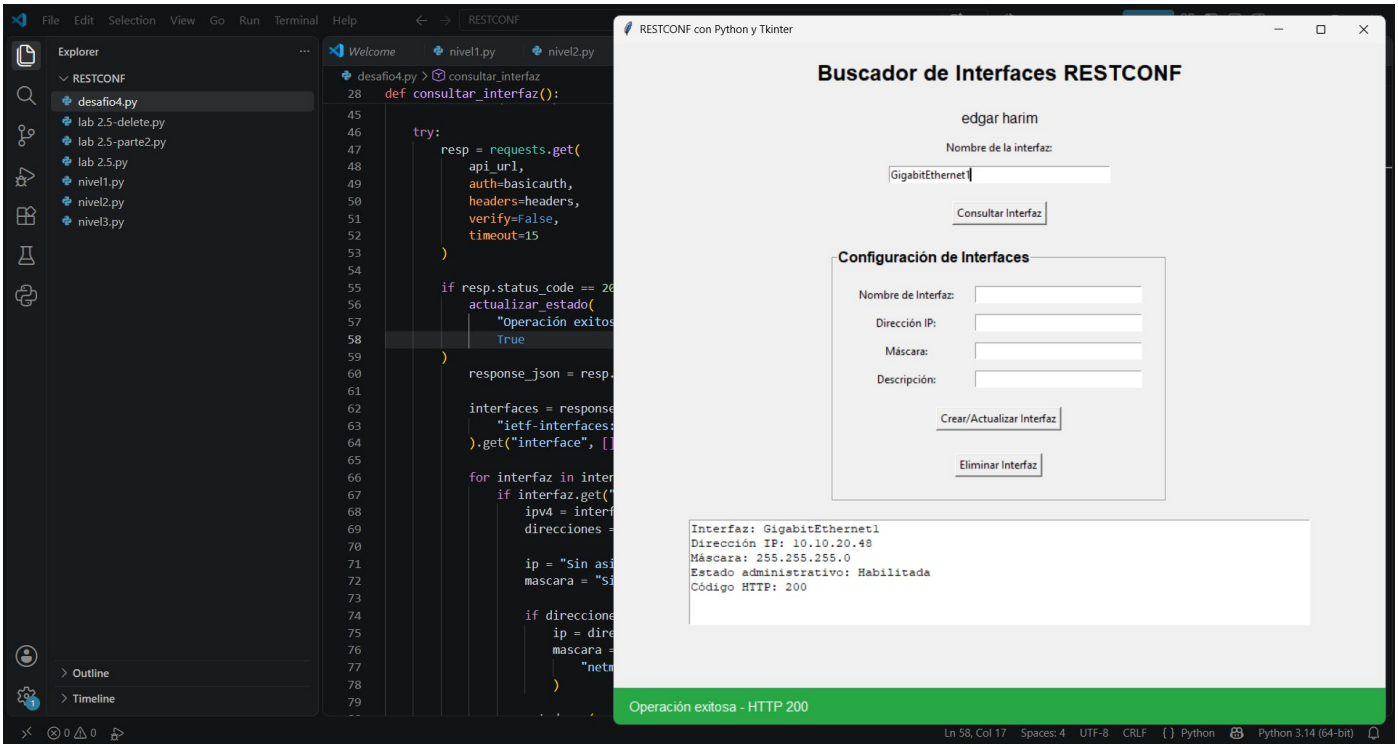
Desafio 4: Indicadores de Estado y Control de Errores

Objetivo: Evitar que la interfaz de Tkinter se congele o colapse ante errores de red o autenticacion.

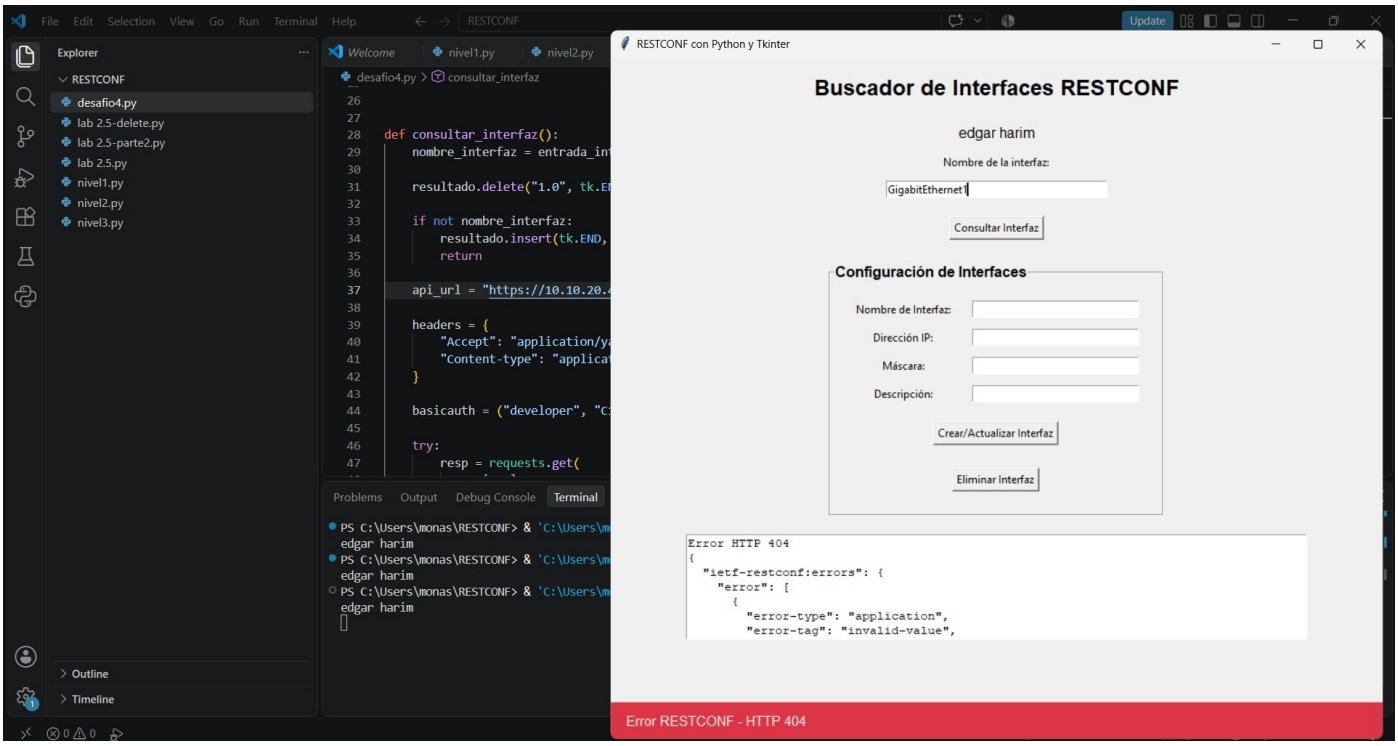
Tarea: Integrar una barra de estado (tk.Label) en la parte inferior de la ventana que cambie de color y texto segun el resultado:

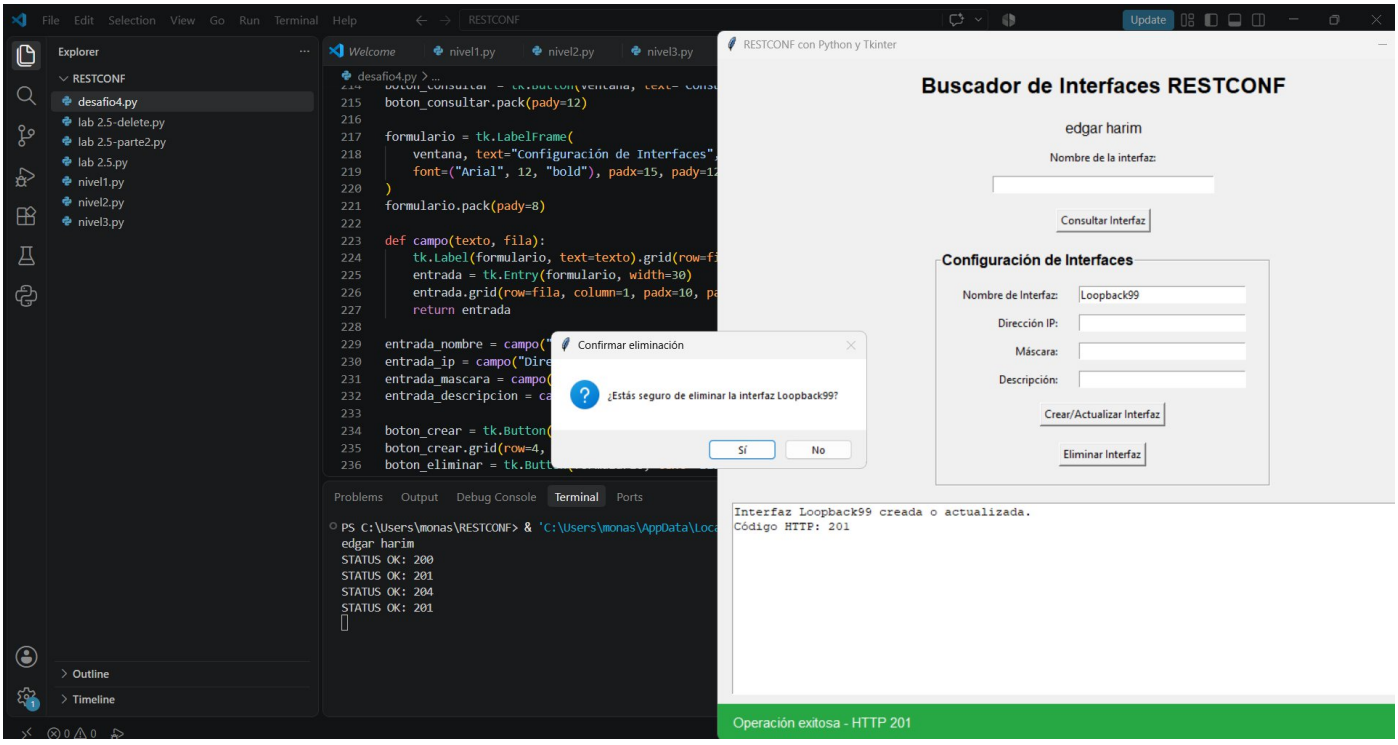
.

- Verde (STATUS 200/201): "Operación exitosa".
- Rojo (Error 401/404/500): Muestra el mensaje de error correspondiente capturado mediante un bloque try-except.



©2017-2026Ciscoand/oritsaffiliates.Allrightsreserved.CiscoPublic





©2017-2026Ciscoand/oritsaffiliates.Allrightsreserved.CiscoPublic

¿Cómo se convierten los datos capturados en la GUI a la estructura de diccionario yangConfig en Python?

Los datos se obtienen de los campos de Tkinter utilizando .get(), despues esos valores se guardan en variables como nombre, IP, mascara y descripción, con ellas se construye el diccionario yangConfig que posteriormente se convierte a json con json.dumps() para enviarlo al router mediante restconf put

¿Cuál es la ventaja de utilizar eventos y funciones manejadoras (callbacks) en Tkinter en lugar de scripts RESTCONF lineales?

La ventaja es que podemos ejecutar las operaciones desde botones, sin tener que modificar y ejecutar el código cada vez, las funciones se activan cuando el usuario presiona un botón, lo que permite consultar, crear y eliminar interfaces desde una misma ventana de forma más sencilla y organizada

©2017-2026Ciscoand/oritsaffiliates.Allrightsreserved.CiscoPublic

Criterio / Indicador	Puntos Posibles	Puntos Obtenidos	Observaciones
1. Estructura Base y Cliente RESTCONF (Python + Tkinter)	**2.5 pts**		
* Creación de la ventana principal de Tkinter, importación de librerías (requests, json, tkinter / ttk) y deshabilitación de advertencias SSL.	0.5		
* Configuración de las peticiones HTTP (GET/PUT) incorporando credenciales *Basic Auth* y encabezados application/yang-data+json desde la GUI.	1.0		
* Consulta exitosa de interfaces mediante botón en la GUI y visualización de la respuesta en los componentes de Tkinter.	1.0		
2. Desafíos Prácticos en la Interfaz Gráfica	**4.0 pts**		
* **Formulario Integrado** (POST/PUT): Captura dinámica de datos (IP, nombre, descripción) desde campos tk.Entry y envío exitoso a RESTCONF (201 Created).	1.5		
* **Eliminación y Cuadros de Diálogo** (DELETE): Implementación de messagebox para confirmar antes de eliminar una interfaz con requests.delete().	1.5		
* **Retroalimentación Visual (UX):** Indicador de estado/mensajes de error en tiempo real (manejo de códigos 200, 401, 404 con try-except).	1.0		
3. Preguntas de Reflexión y Análisis	**2.0 pts**		
* Explica cómo se realiza la conversión de los datos capturados en la GUI (tk.Entry) hacia la estructura de diccionario yangConfig en Python.	1.0		
* Explica la ventaja de utilizar eventos y funciones manejadoras (*callbacks*) en Tkinter al interactuar con servicios web RESTCONF en lugar de scripts lineales.	1.0		
4. Entrega y Flujo en GitHub	**1.5 pts**		
* El Pull Request fue enviado correctamente desde/hacia la rama exacta: feat/unidad4_act02.	0.5		
* El repositorio contiene el archivo de la aplicación de Tkinter (.py) ejecutable y funcional, junto a capturas de la GUI operando.	0.5		
* El Pull Request incluye un título claro y una descripción detallada de la interfaz gráfica desarrollada.	0.5		

Step 2:

©2017-2026Ciscoand/oritsaffiliates.Allrightsreserved.CiscoPublic